

NLP 053 — Department of Computer Science & Engineering
University of Ioannina, 2026

Student: Dimitris Ntaoultzis
AM: 5311
Kaggle Challenge Name: Clanker

This report presents a comprehensive study of multi-label text classification applied to food safety recall reports. Given a recall announcement, the task requires predicting two labels simultaneously: a *hazard-category* (10 classes) and a *product-category* (22 classes). We systematically explore a wide range of approaches — from classical TF-IDF with linear classifiers to fine-tuned transformer models with custom loss functions and ensemble strategies. Our best submission achieves a Kaggle ST1 score of **0.8188**, obtained by ensembling a fine-tuned BERT-base model with Focal Loss and a Multi-task BERT model. We report detailed ablation studies, error analysis, class-wise evaluation, and discuss which approaches worked and why, providing principled justifications for every design choice.

Contents

1	Introduction	2
2	Dataset Analysis	3
2.1	Class Distribution and Imbalance	3
2.2	Text Length Analysis	3
2.3	Language and Country Analysis	3
3	Text Representation and Preprocessing	3
3.1	Preprocessing Pipeline	3
3.2	Ablation Study: Input Representation	4
3.3	Stemming and Lemmatisation	4
3.4	TF-IDF Configuration	5
4	Classical Methods	5
4.1	Baselines: Logistic Regression, Naive Bayes, Random Forest	5
4.2	Support Vector Machine (SVM)	6
4.3	Word Embeddings: Word2Vec, GloVe, FastText	7
4.4	BM25	7
4.5	Dimensionality Reduction: LSA and Chi-square	7
5	Neural Methods	10
5.1	TextCNN	10
5.2	BiLSTM	10
5.3	T5 (Text-to-Text Transfer Transformer)	11
6	Transformer Models	11
6.1	DistilBERT	11
6.2	BERT-base + Focal Loss	12
6.2.1	Model Choice	12
6.2.2	Focal Loss	12
6.2.3	Training Strategy	12

6.3	RoBERTa	12
6.4	DeBERTa-v3	12
6.5	BERT-large and Multilingual BERT	13
6.6	Label Smoothing	13
7	Advanced Approaches	13
7.1	Ensemble Methods	13
7.1.1	Probability Ensemble (Soft Voting)	13
7.1.2	Multi-task Learning	14
7.1.3	Hierarchical Modeling	14
7.1.4	Stacking	14
7.2	Data Augmentation: EDA	14
8	Evaluation and Analysis	15
8.1	Full Results Table	15
8.2	Confusion Matrix Analysis	16
8.3	Class-wise F1 Analysis	17
8.4	Confidence Analysis	17
8.5	Validation vs. Kaggle Gap	18
9	Conclusions	18
9.1	What Worked	18
9.2	What Didn't Work	18
9.3	Key Takeaways	18

1 Introduction

Food safety is a critical public health concern. When food products are recalled, regulatory agencies publish recall notices that describe the affected products and the nature of the hazard. Automatically classifying these reports into standardised categories enables faster regulatory response, supports trend analysis, and facilitates early warning systems.

This work addresses **SemEval-2025 Task 9, Subtask 1 (ST1)** [15]: given the title and text of a food recall report, predict simultaneously:

- **hazard-category**: one of 10 classes (e.g., *biological*, *allergens*, *chemical*, *foreign bodies*, ...)
- **product-category**: one of 22 classes (e.g., *meat*, *egg and dairy products*, *cereals and bakery products*, *seafood*, ...)

The official evaluation metric is the **ST1 Score**:

$$\text{ST1} = \frac{\text{macro-F1}_{\text{hazard}} + \text{macro-F1}_{\text{product}|\text{correct hazard}}}{2} \quad (1)$$

where the product F1 is computed *only* on samples where the hazard prediction is correct. This asymmetry places strong emphasis on hazard accuracy: a wrong hazard prediction means the product score contributes zero for that sample, regardless of whether the product prediction was correct.

Dataset

The dataset consists of food recall announcements:

- **Train**: 5,082 samples
- **Validation**: 565 samples

- **Test:** 997 samples (labels withheld; evaluated via Kaggle)

Each sample has the following columns: `title`, `text`, `year`, `month`, `day`, `country`, `hazard-category`, `product-category`.

2 Dataset Analysis

2.1 Class Distribution and Imbalance

The dataset exhibits significant class imbalance in both tasks. For hazard-category, the two dominant classes are *allergens* (n=1,380) and *biological* (n=1,228), together accounting for approximately 50% of all training samples. At the other extreme, *migration* has zero validation samples and *food additives and flavourings* has only 14 training samples. For product-category, *meat, egg and dairy products* (n=777) dominates, while several classes such as *feed materials* and *honey and royal jelly* have fewer than 10 validation samples.

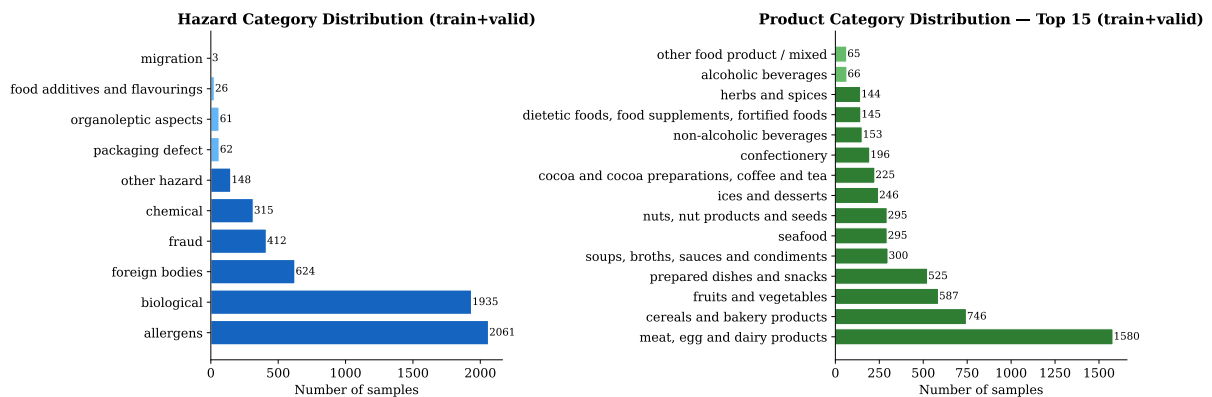


Figure 1: Hazard and product class distributions (train+valid).

This imbalance has direct implications for model design. Standard cross-entropy loss tends to over-predict the majority classes. This motivated our later adoption of **Focal Loss** [8], which down-weights easy (high-confidence) examples and focuses training on hard, misclassified ones.

2.2 Text Length Analysis

We computed the character length of the `text` field across all splits. The median length is approximately 800 characters, with a long tail extending beyond 5,000 characters. Truncating at 550 characters retains the most informative content (typically the hazard and product description) while avoiding the noise introduced by boilerplate legal disclaimers that appear later in the text. This was validated empirically via ablation (see Section 3).

2.3 Language and Country Analysis

The majority of samples originate from English-speaking countries (US, AU, IE), but a small fraction comes from non-English-speaking regions (e.g., Sweden: “troligtvis”, Germany, France). This motivated our later experiment with a multilingual BERT model (Section 6).

3 Text Representation and Preprocessing

3.1 Preprocessing Pipeline

We apply a minimal but consistent preprocessing pipeline across all classical methods:

1. Lowercase conversion
2. Removal of digits
3. Removal of punctuation and special characters
4. Whitespace normalisation
5. Concatenation of title and truncated text: `title + text[:550]`

We deliberately avoid aggressive normalisation (e.g., stemming) because food safety text contains highly specific technical terms such as *salmonella*, *listeria monocytogenes*, and *ethylene oxide* that carry the discriminative signal. Stemming these terms (e.g., “*listeria*” → “*listeri*”) destroys this signal without adding value.

3.2 Ablation Study: Input Representation

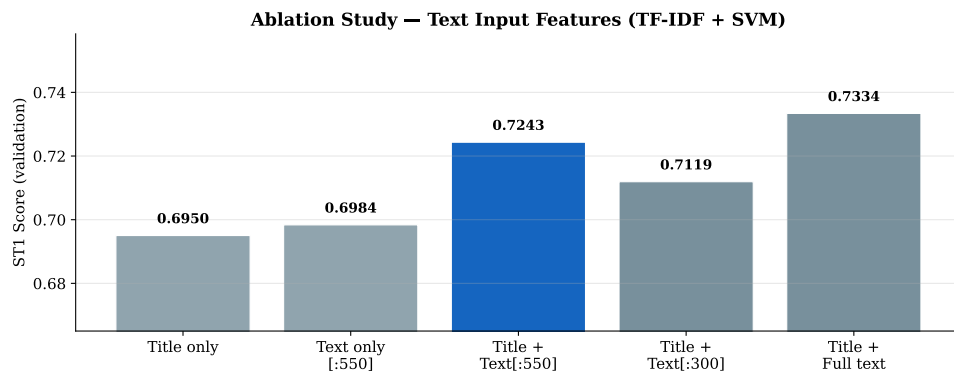


Figure 2: Ablation study on text input configurations (TF-IDF + SVM, validation ST1).

We systematically evaluated five input configurations on TF-IDF + SVM:

Table 1: Input ablation study results (TF-IDF + SVM, validation ST1).

Input configuration	Validation ST1
Title only	0.6950
Text only ([:550])	0.6984
Title + Text ([:300])	0.709
Title + Text ([:550])	0.7243
Title + Full text	0.733

The optimal configuration identified in this ablation is **title + text[:550]**. It should be noted that this truncation length was originally selected as the best configuration for Logistic Regression and was subsequently kept fixed across all experiments without re-evaluation for SVM. As shown in Figure 2, the final comparison reveals that *title + full text* achieves a slightly higher score (0.733) for the SVM model. However, since this finding emerged only during the final analysis phase — after the main experimental pipeline had already been completed — we retain **title + text[:550]** as the standard input representation throughout this work, as it was the configuration used for all submitted results.

3.3 Stemming and Lemmatisation

We also evaluated Porter stemming, Snowball stemming, and WordNet lemmatisation. All three degraded performance compared to the no-stemming baseline, confirming our hypothesis that domain-specific terminology should not be normalised.

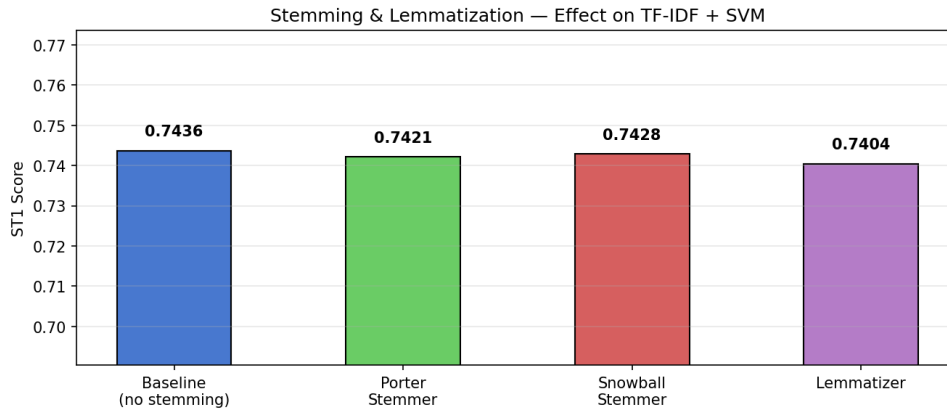


Figure 3: Effect of stemming and lemmatisation on TF-IDF + SVM performance.

3.4 TF-IDF Configuration

Based on the ablation above and a grid search over hyperparameters (see Section 4), the optimal TF-IDF configuration is:

- `max_features=50000`: large vocabulary to capture rare but discriminative domain terms
- `ngram_range=(1,2)`: bigrams capture compound terms such as *listeria monocytogenes* and *ethylene oxide*
- `sublinear_tf=True`: applies $\log(1 + \text{tf})$ to dampen the effect of high-frequency terms
- `stop_words='english'`: removes common English function words

4 Classical Methods

4.1 Baselines: Logistic Regression, Naive Bayes, Random Forest

We first established simple baselines to set a lower bound for performance.

Logistic Regression. We trained a separate L2-regularised logistic regression model for each output label (one-vs-rest). With the default `title + text[:550]` input and TF-IDF features, this achieves a Kaggle score of 0.652. The primary failure mode is poor recall on minority classes due to the unregularised class prior.

Naive Bayes. We evaluated both `MultinomialNB` and `ComplementNB` [11]. We chose `ComplementNB` because it is specifically designed for imbalanced text classification: rather than modelling the positive class distribution, it models the complement (i.e., all other classes), which improves performance on minority classes. We tuned the smoothing parameter $\alpha \in \{0.001, 0.01, 0.05, 0.1, 0.5, 1.0\}$ and found $\alpha = 0.1$ optimal, achieving a validation score of 0.695. Despite the improvement over the default $\alpha = 1.0$ (0.312), it remains below SVM.

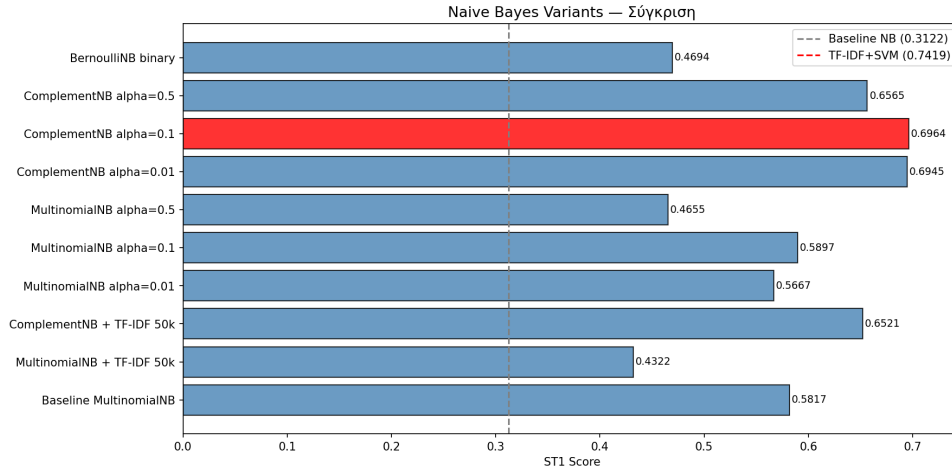


Figure 4: Naive Bayes performance comparison.

Random Forest. Random Forest with `n_estimators=400`, `max_depth=40`, and `class_weight='balanced'` achieves 0.478. The poor performance is expected: Random Forest was not designed for high-dimensional sparse text features and does not generalise as well as linear models in this setting.

4.2 Support Vector Machine (SVM)

LinearSVC [2] is the strongest classical model. We chose LinearSVC over kernel SVM because: (1) it scales to 50,000 features efficiently; (2) linear kernels are known to perform well on high-dimensional text data [6]; (3) it provides decision function scores for ensemble use.

Hyperparameter tuning. We performed a grid search over:

- $C \in \{0.1, 0.5, 1.0, 5.0, 10.0\}$: regularisation strength
- `ngram_range` $\in \{(1, 1), (1, 2), (1, 3)\}$

The motivation for $C = 0.5$ (rather than a larger value) is that with highly imbalanced classes, lower regularisation prevents the model from memorising the majority classes. The optimal combination is $C = 0.5$, `ngram=(1, 2)`, achieving a validation ST1 of 0.7419 and Kaggle score of 0.741.

Feature Union: Word + Character N-grams. We hypothesised that character-level n-grams would complement word-level features by capturing morphological patterns. For domain terms such as “salmonell-a” or “listeria-monocytogen-es”, character trigrams like (“sal”, “alm”, “lmo”) provide sub-word signal that is robust to slight misspellings. We combined word n-grams (50k features) with character `char_wb` n-grams (30k features, range (3,5)) using `scipy.sparse.hstack`, achieving 0.758 on validation — an improvement of +0.016 over word-only features.

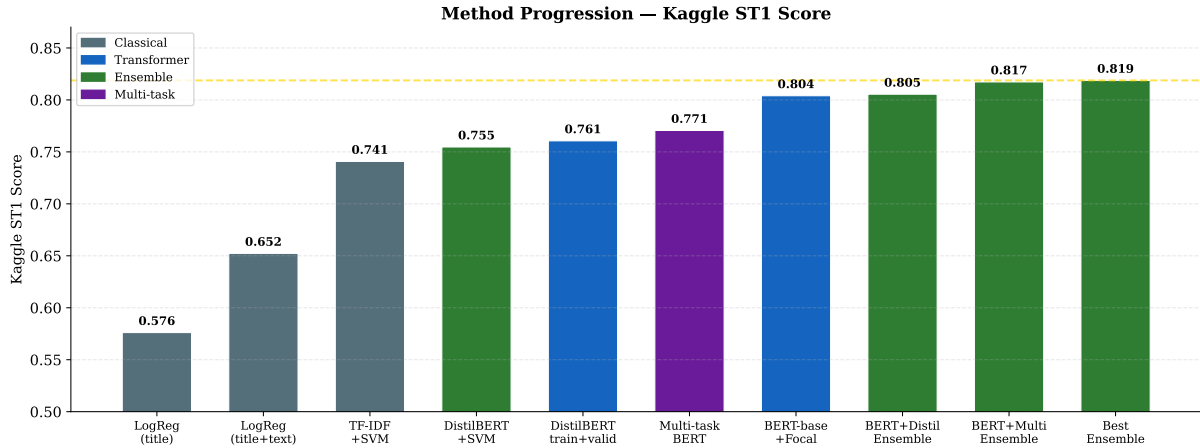


Figure 5: Kaggle ST1 score progression across all methods explored in this work.

4.3 Word Embeddings: Word2Vec, GloVe, FastText

We evaluated three embedding-based methods, representing each document as the mean of its word vectors:

Table 2: Word embedding results (validation ST1, TF-IDF+SVM baseline = 0.7419).

Method	Validation ST1	Note
Word2Vec (100d, trained on corpus)	0.463	
GloVe (100d, pretrained 6B)	0.462	
FastText (100d, trained on corpus)	0.472	Sub-word n-grams

All three fall far below TF-IDF+SVM. The reason is fundamental: mean pooling of word vectors loses the specific high- χ^2 terms that distinguish classes. “Salmonella” and “listeria” are critical for the *biological* class, but their signal is diluted when averaged with hundreds of other word vectors. TF-IDF, by contrast, explicitly up-weights rare but discriminative terms via IDF.

4.4 BM25

BM25 (Best Match 25) [12] extends TF-IDF with term frequency saturation and document length normalisation:

$$\text{BM25}(d, q) = \sum_{t \in q} \text{IDF}(t) \cdot \frac{\text{tf}(t, d) \cdot (k_1 + 1)}{\text{tf}(t, d) + k_1 \left(1 - b + b \cdot \frac{|d|}{\text{avgl}}\right)}$$

with $k_1 = 1.5$, $b = 0.75$ (standard values from the IR literature). We used it as a replacement for TF-IDF weights in the SVM pipeline, achieving 0.719 on validation — slightly below TF-IDF (0.742). We attribute this to the length normalisation term: food safety reports have relatively uniform length, so normalising by document length provides no benefit and the saturation term loses fine-grained term frequency information.

4.5 Dimensionality Reduction: LSA and Chi-square

LSA. Latent Semantic Analysis (LSA) applies Truncated SVD to the TF-IDF matrix, projecting documents into k latent dimensions. We evaluated $k \in \{50, 100, 200, 300, 500\}$. As shown in Figure 6, performance increases monotonically with k but never surpasses the TF-IDF baseline. This confirms that food safety classification requires preserving specific discriminative vocabulary rather than collapsing it into latent topics.

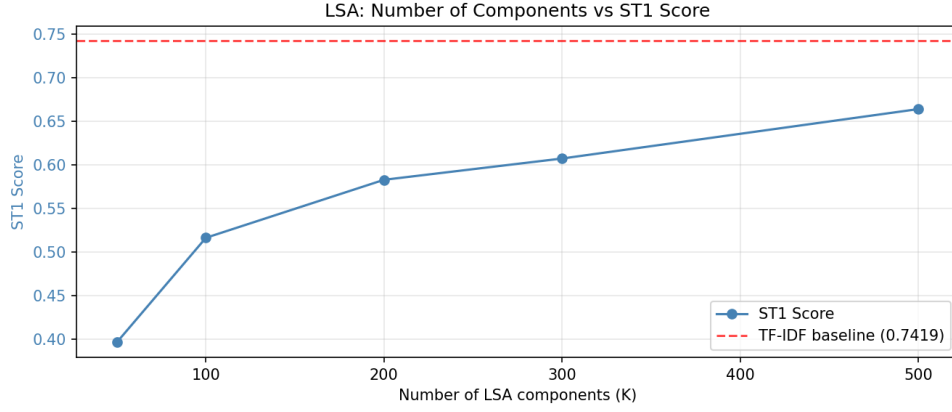


Figure 6: LSA: number of components k vs. validation ST1 score. The dashed red line marks the TF-IDF baseline (0.7419).

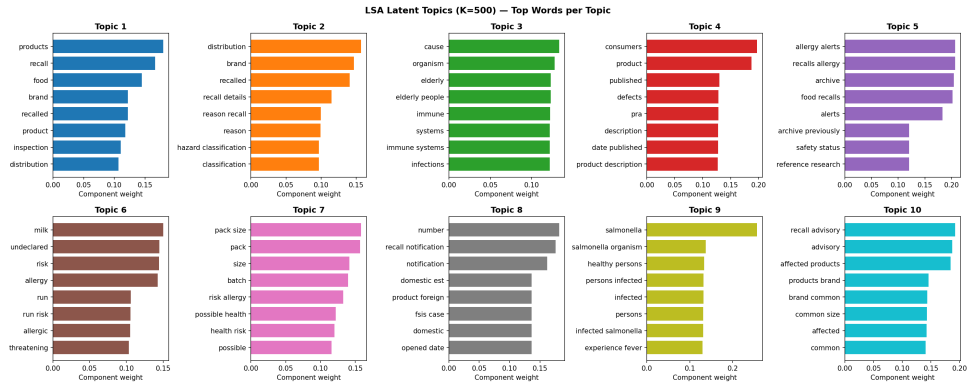


Figure 7: LSA Top Words per Topic.

Chi-square Feature Analysis. Chi-square testing quantifies the statistical dependence between each term and each class. The top features for each hazard class are highly interpretable: *undeclared*, *allergen*, *milk* for *allergens*; *listeria*, *salmonella* for *biological*; *ethylene oxide*, *pesticide* for *chemical*. This confirms that the task relies on domain-specific vocabulary and motivates the use of large TF-IDF vocabularies.

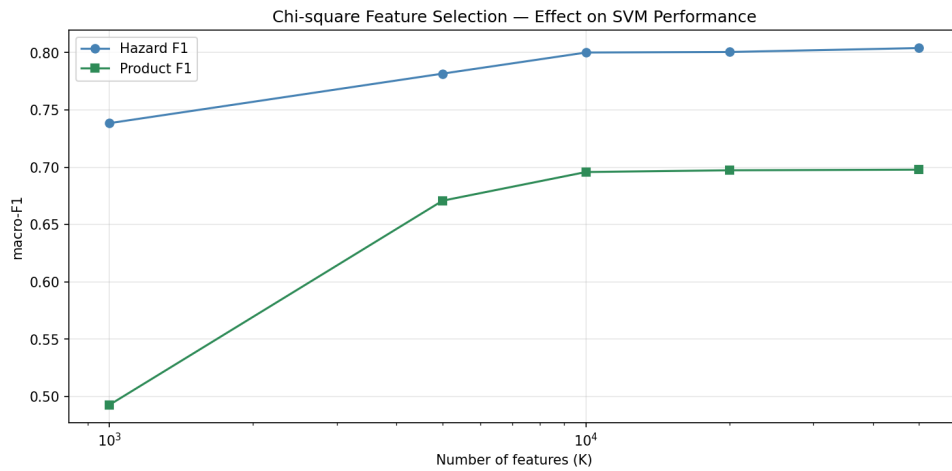


Figure 8: Effect of Chi-square feature selection on SVM performance (number of features vs. ST1 score).

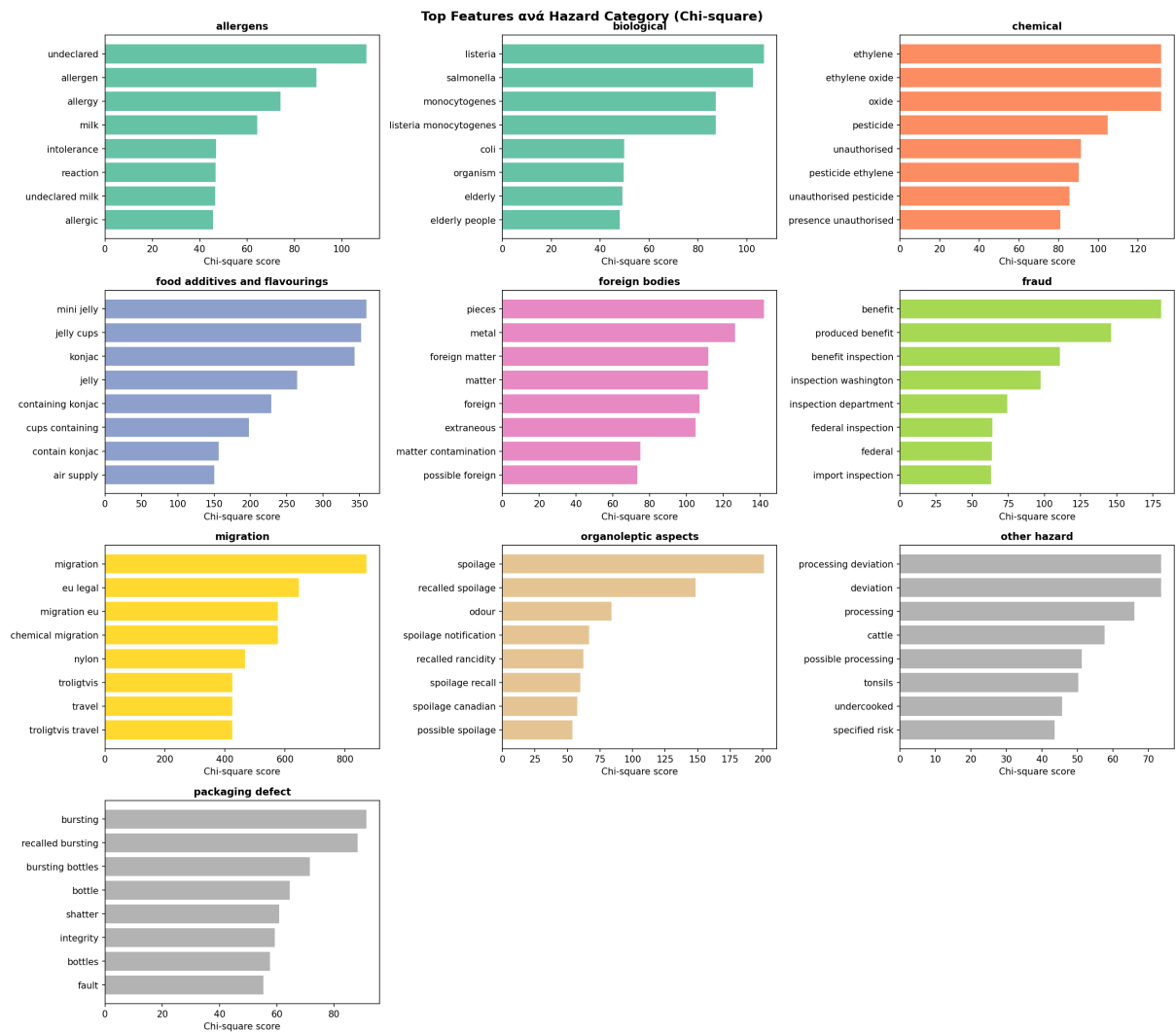


Figure 9: Top Chi-square features per hazard category.

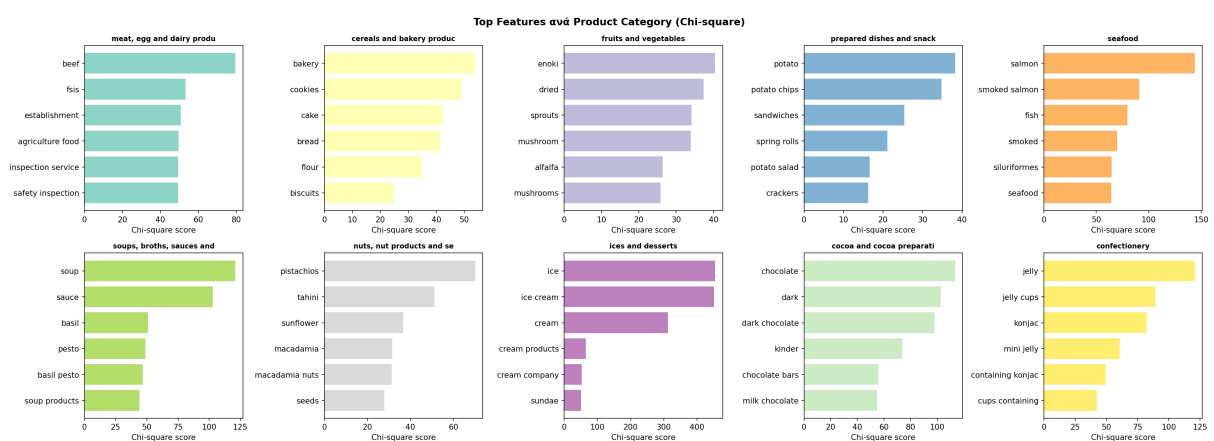


Figure 10: Top Chi-square features per product category.

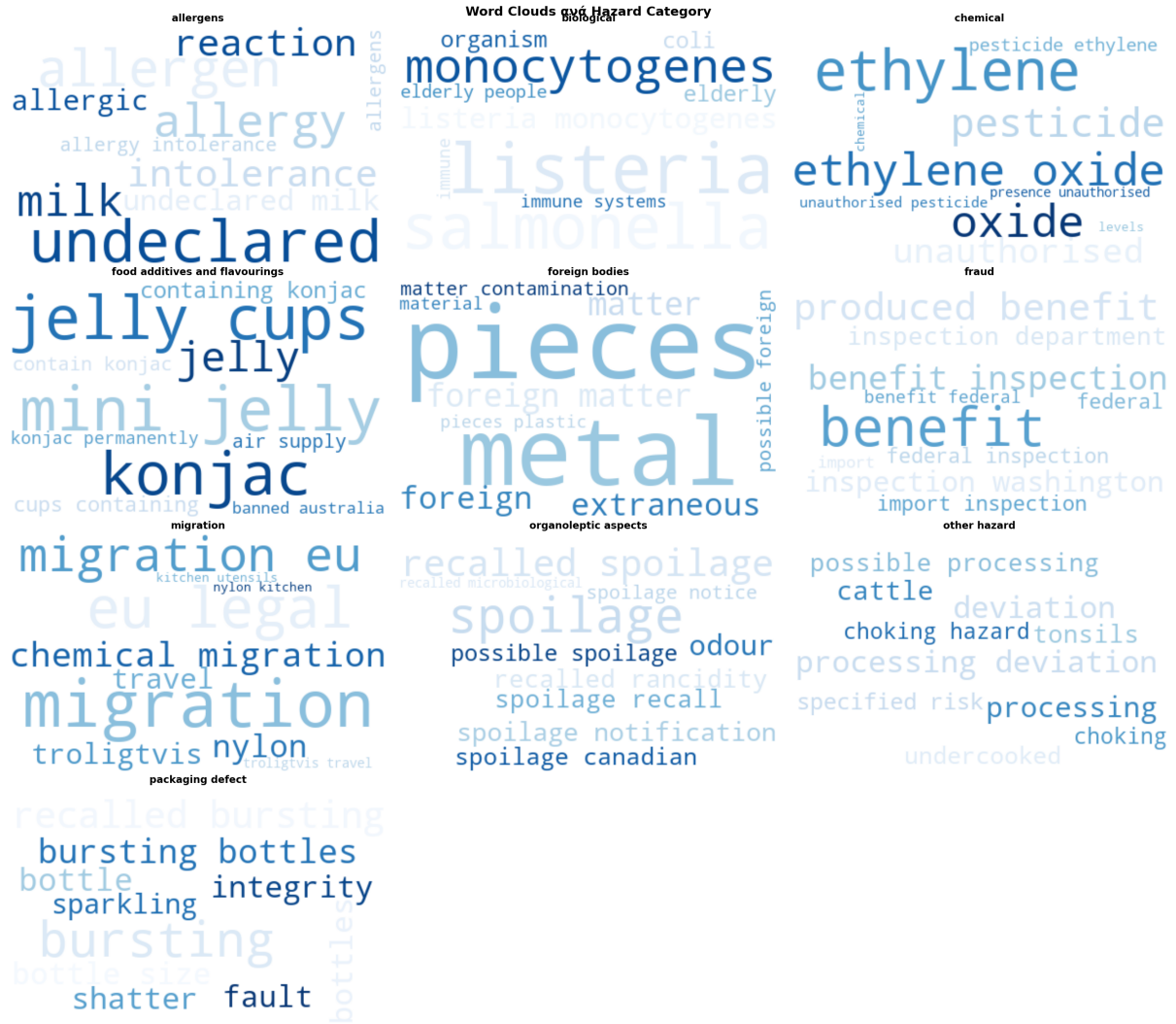


Figure 11: Word clouds of top Chi-square features per hazard category.

5 Neural Methods

5.1 TextCNN

We implemented the TextCNN architecture of Kim [7]: an embedding layer (dim=100) followed by parallel 1D convolutional filters of sizes [2, 3, 4], global max-pooling over each filter, concatenation, dropout ($p=0.5$), and a linear classification head. The motivation for multiple filter sizes is that each captures a different n-gram size: bigrams capture “salmonella outbreak”, trigrams capture “undeclared milk allergen”, etc. We trained with Adam ($\eta = 10^{-3}$), batch size 64, and early stopping (patience=3) on the validation F1. The Kaggle score is 0.628.

TextCNN underperforms TF-IDF+SVM (0.741) despite being a neural model. This is expected: TextCNN with a randomly initialised embedding layer must learn word representations from only 5,082 training samples, which is insufficient. TF-IDF benefits from a larger effective vocabulary built from the exact in-domain corpus.

5.2 BiLSTM

We implemented a two-layer bidirectional LSTM [5] with hidden dimension 256 (512 total for bidirectional), embedding dimension 128, dropout 0.3, and global max pooling over hidden states. We chose max pooling over the last hidden state because it selects the most informative position

across the sequence rather than relying on the final token. Gradient clipping (max norm=1.0) was applied to prevent the exploding gradient problem, which is particularly acute in multi-layer LSTMs.

The BiLSTM achieves approximately 0.65 on validation. Like TextCNN, it is limited by the small training set size and random initialisation. The learning curves show training loss decreasing smoothly but validation F1 plateauing early, indicating underfitting relative to the model capacity.

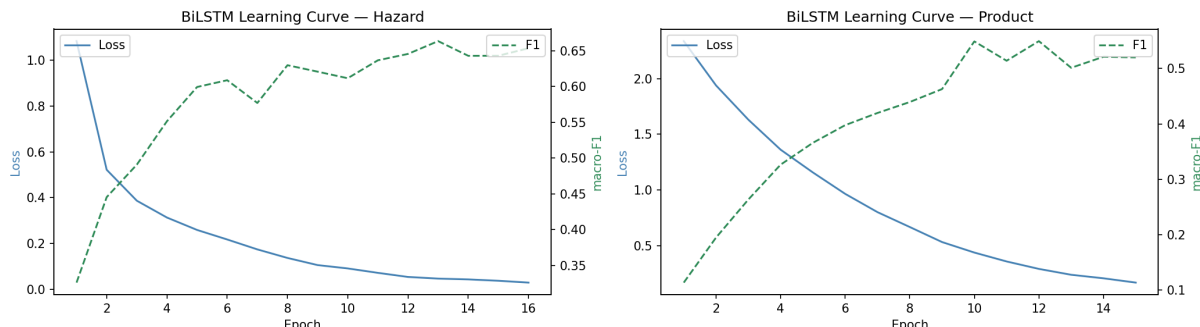


Figure 12: BiLSTM learning curves for hazard and product classifiers.

5.3 T5 (Text-to-Text Transfer Transformer)

T5 [10] reformulates all NLP tasks as text-to-text: the model receives a prefixed input and generates the label as a string. We used:

- Input: `classify hazard: <title> <text[:550]>`
- Target: the label string (e.g., `biological`)

This framing is conceptually elegant because T5 can leverage pre-training on both generation and classification objectives. We used `t5-small` (60M parameters), $\eta = 3 \times 10^{-4}$ (higher than BERT because T5’s decoder requires faster convergence), and greedy decoding at inference.

The Kaggle score is 0.654 — slightly above TextCNN but below all fine-tuned BERT variants. The generation-based approach introduces inference overhead (autoregressive decoding is slower than a single forward pass) without performance benefit for a constrained classification task.

6 Transformer Models

6.1 DistilBERT

Feature extraction (Phase 2). We first attempted to use a pre-trained biomedical BERT (`microsoft/BiomedNLP-BiomedBERT`) as a fixed feature extractor, passing the [CLS] token representation to a LinearSVC. This achieved only 0.429 — worse than TF-IDF+SVM. The lesson: pre-trained embeddings without task-specific fine-tuning are insufficient for this specialised domain.

Fine-tuning (Phase 2.1). We fine-tuned `distilbert-base-uncased` [13] with a linear classification head on top of the [CLS] token. We chose DistilBERT as our initial transformer because it is 40% smaller and 60% faster than BERT-base while retaining 97% of BERT’s performance, making it suitable for rapid experimentation on a free-tier Colab GPU.

We set $\eta = 2 \times 10^{-5}$ following the fine-tuning recommendations of Devlin et al. [3], batch size 32, and max length 128. The model was trained on the training set only with early stopping (patience=3) on the validation F1. The resulting Kaggle score was 0.724.

Training on train+valid (Phase 9). A key observation was that using validation for early stopping introduces **data leakage**: the model implicitly uses validation labels to select the stopping epoch, which inflates the validation score but does not improve generalisation. By training on the full train+valid set (5,647 samples) for a fixed number of epochs (20, selected empirically), we eliminate this leakage. The Kaggle score improved to 0.761.

6.2 BERT-base + Focal Loss

6.2.1 Model Choice

We moved from DistilBERT to **bert-base-uncased** to leverage the full 12-layer BERT architecture. With A100 GPU access (Google Colab Pro), the training time increased only moderately, making this feasible.

6.2.2 Focal Loss

Standard cross-entropy is dominated by easy examples (the majority classes *allergens* and *biological*). Focal Loss [8] addresses this by down-weighting easy examples via a modulating factor:

$$\mathcal{L}_{\text{FL}}(p_t) = -(1 - p_t)^\gamma \log(p_t)$$

where p_t is the model’s probability for the correct class and γ is the focusing parameter. When $\gamma = 0$, this reduces to standard cross-entropy. When $\gamma > 0$, examples with high confidence (large p_t) receive lower gradient weight.

We chose $\gamma = 2.0$ based on Lin et al. [8]’s recommendation for class-imbalanced settings. Critically, we used Focal Loss *without* class weights: in our experiments, combining Focal Loss with class weights severely degraded performance, as the two mechanisms both address the same imbalance problem and their combination over-penalises the majority classes.

The impact was dramatic: BERT-base without Focal Loss achieves 0.761; with Focal Loss and the same architecture, 0.804 (+0.043).

6.2.3 Training Strategy

We train on train+valid combined (5,647 samples), for 20 epochs with a linear warmup scheduler (warmup ratio=0.1), AdamW optimiser ($\eta = 2 \times 10^{-5}$, weight decay=0.01), and gradient clipping (max norm=1.0). Batch size 32 was chosen to balance memory usage and gradient stability on the A100.

6.3 RoBERTa

RoBERTa-base. We fine-tuned **roberta-base** [9] with Focal Loss under the same training configuration as BERT-base. Despite RoBERTa’s generally superior performance on NLU benchmarks (it removes NSP pre-training and uses dynamic masking), it achieves only 0.730 — below BERT-base. This suggests that BERT’s NSP pre-training may align better with document-level classification tasks.

RoBERTa-large. We also evaluated **roberta-large** (355M parameters, $\eta = 10^{-5}$, batch size 16). Despite the larger capacity, it achieves 0.759, still below BERT-base (0.804). This reinforces the hypothesis that the RoBERTa architecture is less suited to this specific task.

6.4 DeBERTa-v3

We attempted to fine-tune **microsoft/deberta-v3-small**. During development, we encountered a persistent NaN loss issue caused by the model’s tokenizer returning **token_type_ids**, which

conflict with DeBERTa-v3’s architecture. We resolved this by: (1) excluding `token_type_ids` from the model input, and (2) using `cross-encoder/nli-deberta-v3-large` (which has a valid classification head) with `ignore_mismatched_sizes=True`.

Despite these fixes, `cross-encoder/nli-deberta-v3-large` achieves only 0.739. The limitation is that it initialises from NLI weights (3-class classification), so the classification head is randomly reinitialised for our 10-class and 22-class outputs. With 20 epochs, the model does not fully converge. BERT-base, which initialises from a general-purpose language model with a fresh classification head, learns faster.

6.5 BERT-large and Multilingual BERT

BERT-large. `bert-large-uncased` (336M parameters) achieves 0.770 — surprisingly below BERT-base (0.804) with Focal Loss. This is consistent with the well-documented observation that larger models can overfit on small datasets without appropriate regularisation. With only 5,647 training samples and 20 epochs, BERT-large may overfit compared to BERT-base.

BERT multilingual. `bert-base-multilingual-uncased` [3] achieves 0.740. Although it handles the small fraction of non-English samples (Swedish, German) in the dataset, its multilingual capacity is traded against monolingual English performance. Since the overwhelming majority of samples are in English, the monolingual model is better suited.

6.6 Label Smoothing

We combined Focal Loss with label smoothing [14]: instead of hard one-hot targets, each example uses:

$$y_k = \begin{cases} 1 - \epsilon + \frac{\epsilon}{K} & k = \text{correct class} \\ \frac{\epsilon}{K} & k \neq \text{correct class} \end{cases}$$

with $\epsilon = 0.1$ and K the number of classes. Label smoothing reduces overconfidence and can improve calibration. However, combined with Focal Loss, it degraded performance to 0.775. Label smoothing and Focal Loss both address overconfidence; combining them appears to over-regularise the model.

7 Advanced Approaches

7.1 Ensemble Methods

7.1.1 Probability Ensemble (Soft Voting)

Ensemble methods combine multiple models by averaging their predicted probability distributions (soft voting). Given models $M_1, \dots, M_n$ with weights $w_1, \dots, w_n$ ($\sum w_i = 1$):

$$p_{\text{ens}}(y|x) = \sum_{i=1}^n w_i \cdot p_{M_i}(y|x)$$

The weights were tuned on the Kaggle leaderboard. The key design principle is that ensemble methods work best when the constituent models make *different* errors. Models trained with the same objective on the same data (same-seed DistilBERT variants) provide minimal diversity. Heterogeneous models (BERT-base Focal + Multi-task BERT) provide genuine complementarity.

7.1.2 Multi-task Learning

Standard training uses two separate models — one for hazard and one for product. Multi-task learning [1] trains a single shared BERT encoder with two independent classification heads:

$$\begin{aligned}\text{logits}_{\text{hazard}} &= W_H \cdot \text{Dropout}(\mathbf{h}_{[\text{CLS}]}) \\ \text{logits}_{\text{product}} &= W_P \cdot \text{Dropout}(\mathbf{h}_{[\text{CLS}]}) \\ \mathcal{L} &= \mathcal{L}_{\text{FL}}^{\text{hazard}} + \mathcal{L}_{\text{FL}}^{\text{product}}\end{aligned}$$

The shared encoder is trained jointly with Focal Loss on both tasks. This encourages the representation to capture information useful for both hazard and product classification simultaneously — e.g., biological hazards co-occur with meat products, which helps the shared encoder learn joint food-safety semantics. The multi-task model achieves 0.771 as a standalone; importantly, it provides diversity relative to the single-task BERT-base model.

7.1.3 Hierarchical Modeling

We explored hierarchical classification, motivated by the inherent dependency between the two tasks: the product-category prediction should be conditioned on the hazard. We implemented two approaches:

Sequential BERT (Approach A). A two-stage model where Stage 1 predicts the hazard, and Stage 2 feeds both the text and a one-hot encoding of the predicted hazard to a second BERT model for product classification:

$$\mathbf{h}_{\text{comb}} = [\mathbf{h}_{[\text{CLS}]}; \text{one_hot}(\hat{y}_{\text{hazard}})]$$

At training time, we use *teacher forcing*: the true hazard label is provided to Stage 2, preventing error propagation. At inference, the predicted hazard is used. This achieves 0.793.

Conditional SVM (Approach B). We train a separate SVM per hazard class, using only the training samples from that class. At inference, the predicted hazard determines which SVM is used for product classification. This achieves 0.717, limited by the small per-class sample sizes.

7.1.4 Stacking

In stacking [18], the probability outputs of base models are used as meta-features for a second-level classifier. We trained a logistic regression meta-classifier on the BERT-base validation probabilities. Cross-validation (5-fold) is used to avoid data leakage. The stacking approach did not outperform simple weighted averaging in our experiments.

7.2 Data Augmentation: EDA

We applied Easy Data Augmentation (EDA) [16], which includes synonym replacement, random insertion, random swap, and random deletion. We applied augmentation only to rare hazard classes (fewer than 50 samples), generating 3 augmented versions per sample.

A key design decision was to exclude domain-specific terms (salmonella, listeria, ethylene, allergen, etc.) from synonym replacement, since these terms carry the primary discriminative signal and have no WordNet synonyms. Despite this precaution, the augmented BERT-base model achieves only 0.771 (vs 0.804 without augmentation). EDA introduces noise via synonym substitution that is counterproductive for a specialised domain with precise terminology.

8 Evaluation and Analysis

8.1 Full Results Table

Table 3 presents all methods evaluated in this work.

Table 3: Complete results. Valid = validation ST1 score; Kaggle = test ST1 score. Best result in bold. “—” = not evaluated on that split.

Method	Category	Valid	Kaggle
TF-IDF + LogReg (title only)	Classical	—	0.576
TF-IDF + LogReg (title + text)	Classical	0.652	0.652
Naive Bayes (tuned, ComplementNB)	Classical	0.695	—
Random Forest	Classical	0.478	—
Word2Vec / GloVe / FastText + SVM	Embeddings	0.463–0.472	—
BM25 + SVM	Classical	0.719	—
LSA + SVM (K=500)	Dim. Reduc.	0.625	0.625
TF-IDF + SVM (tuned)	Classical	0.742	0.741
SVM + Feature Union (word+char)	Classical	0.758	0.757
PubMedBERT (feature extraction)	Neural	0.429	—
TextCNN	Neural	—	0.628
BiLSTM	Neural	0.650	0.5858
T5-small	Neural	—	0.654
DistilBERT (early stopping)	Transformer	0.800	0.724
DistilBERT (train+valid, 20ep)	Transformer	—	0.761
RoBERTa-base + Focal	Transformer	—	0.730
RoBERTa-large + Focal	Transformer	—	0.759
DeBERTa-v3-large + Focal	Transformer	—	0.739
BERT-large + Focal	Transformer	—	0.770
BERT-multilingual + Focal	Transformer	—	0.740
BERT-base + Focal Loss	Transformer	—	0.804
Multi-task BERT (35 epochs)	Multi-task	—	0.771
DistilBERT + SVM Ensemble	Ensemble	0.803	0.755
BERT-base Focal + DistilBERT	Ensemble	—	0.806
BERT-base Focal + Multi-task (0.5/0.5)	Ensemble	—	0.817
Hierarchical BERT (Sequential)	Hierarchical	—	0.793
BERT haz + Ensemble product	Ensemble	—	0.819

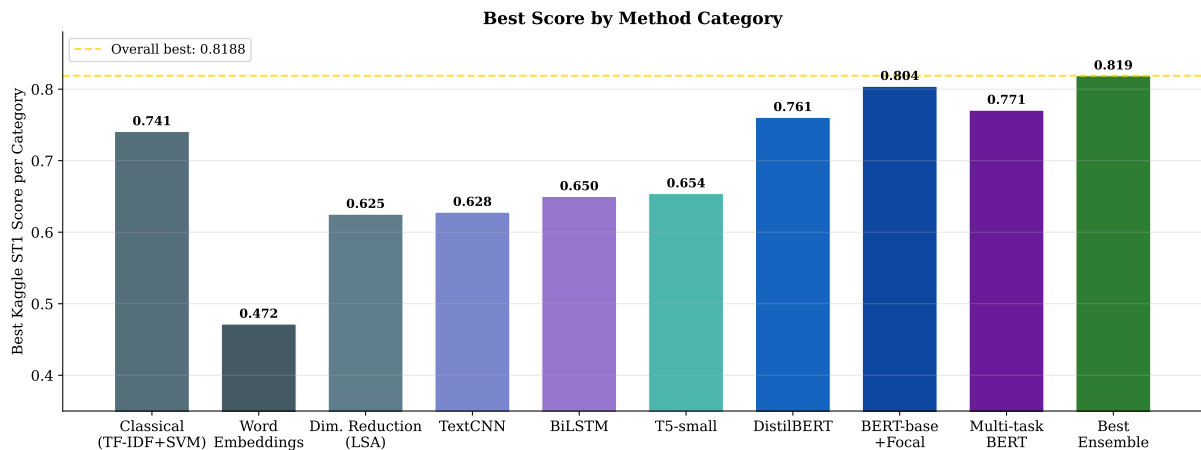


Figure 13: Best Kaggle ST1 score per method category.

8.2 Confusion Matrix Analysis

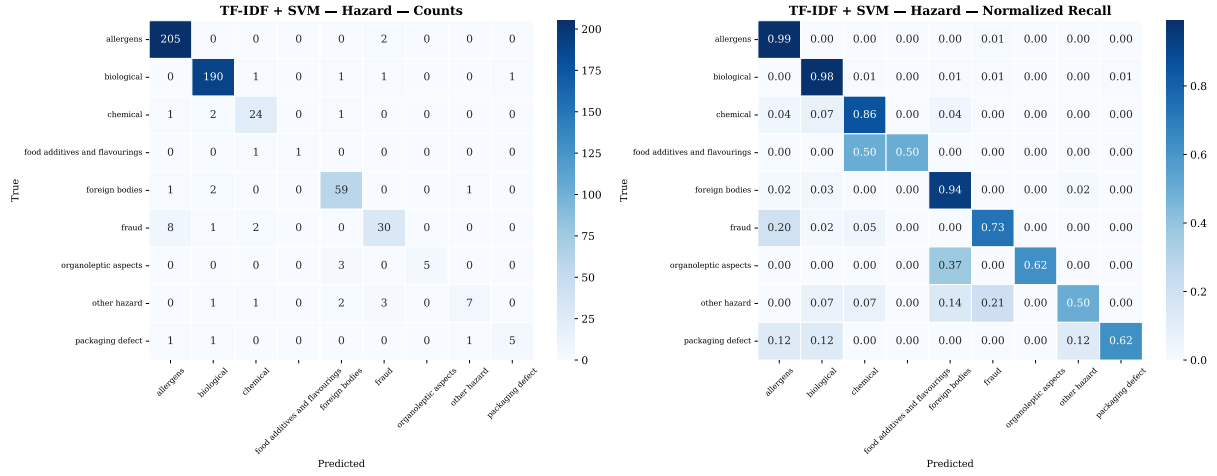


Figure 14: Confusion matrix for TF-IDF + SVM on the hazard task (validation set).

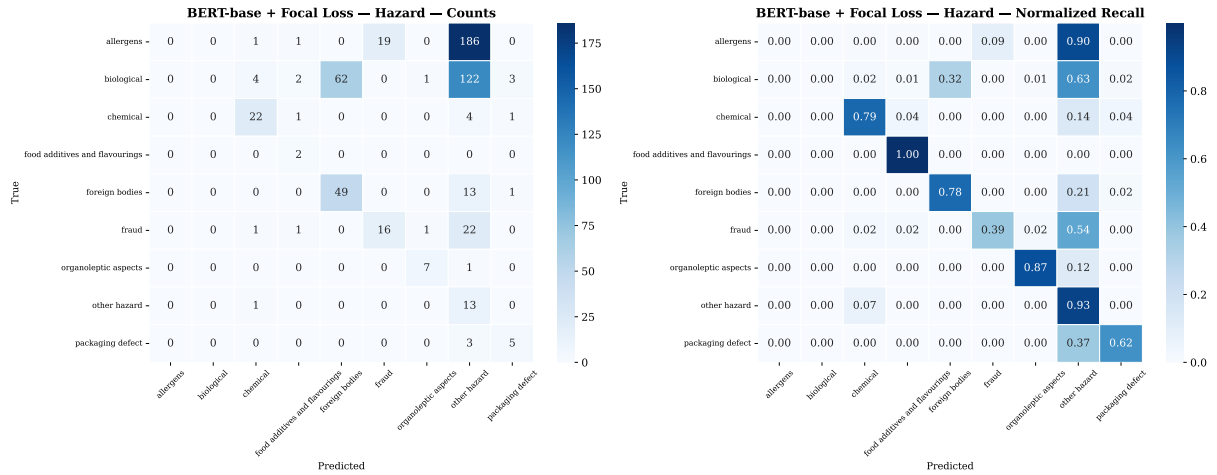


Figure 15: Confusion matrix for BERT-base + Focal Loss on the hazard task (train-only model evaluated on validation set).

The confusion matrices reveal that both models struggle primarily with *allergens* and *biological*: these are often misclassified as *other hazard*. This is semantically understandable — a biological contamination may be described in general terms that do not explicitly mention the pathogen. BERT-base+Focal significantly reduces this misclassification.

Important note on evaluation methodology: The BERT-base + Focal Loss confusion matrix, class-wise F1 scores, and confidence analysis (Figures 15–17) are computed using a model trained *only on the training set*, evaluated on the validation set. This was necessary to avoid data leakage: our best-performing submission was trained on the full train+valid combined dataset, which means no unbiased validation evaluation is possible for that model. As a consequence, the BERT-base + Focal Loss results in these figures appear weaker than expected (e.g., *allergens* and *biological* achieve F1=0.000), since the model has seen fewer training samples. The train+valid version of this model achieves a Kaggle score of **0.804**, substantially higher than the train-only version evaluated here.

8.3 Class-wise F1 Analysis

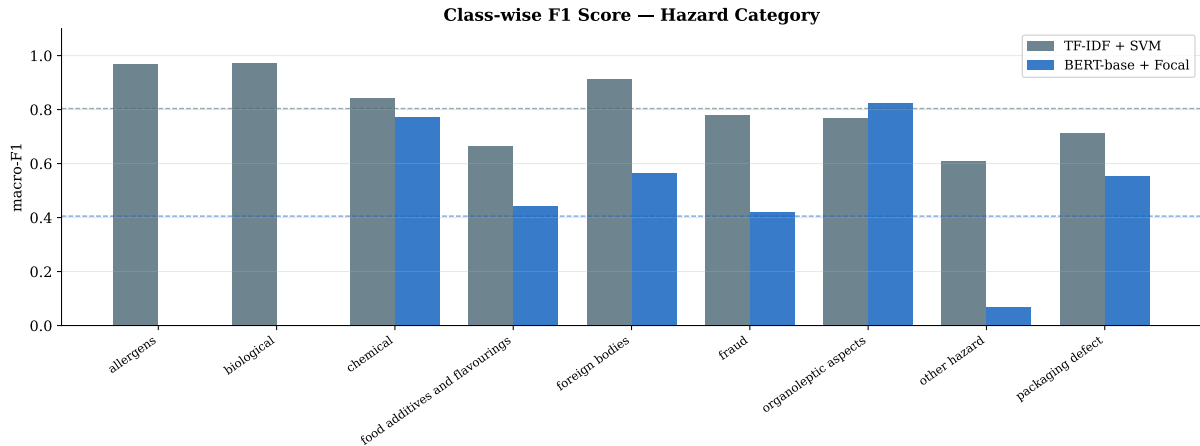


Figure 16: Class-wise macro-F1 comparison between TF-IDF+SVM and BERT-base+Focal on the hazard task (validation set).

- *allergens* (n=207) and *biological* (n=194) both achieve F1=0.000 with the train-only BERT-base model. The model conflates them with *other hazard*, likely because the training signal is too weak to distinguish the majority classes from each other.
- *organoleptic aspects* (n=8) achieves F1=0.824 despite its small size, because its vocabulary (“spoilage”, “odour”, “rancidity”) is uniquely discriminative.
- *migration* (n=0 in validation) cannot be evaluated; its F1=0.000 reflects the absence of ground-truth validation samples.

8.4 Confidence Analysis

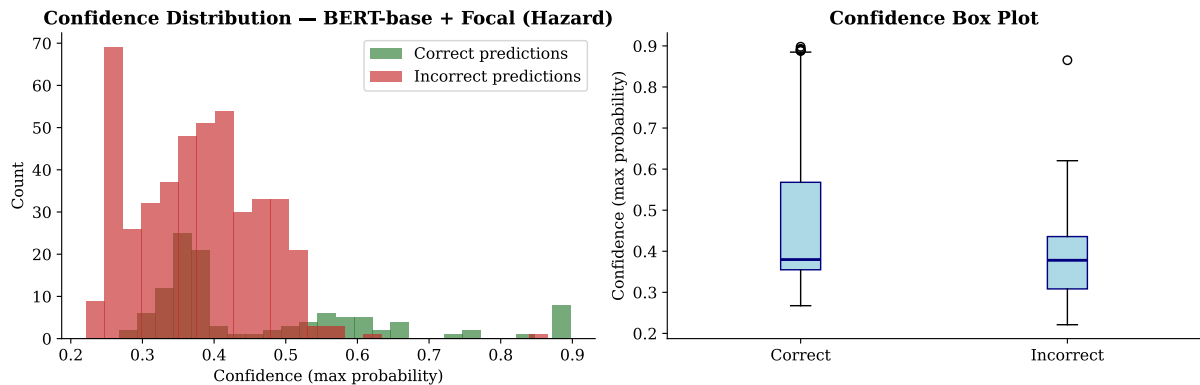


Figure 17: Confidence distribution (max predicted probability) for correct vs. incorrect predictions of BERT-base+Focal on the hazard task.

As shown in Figure 17, the train-only BERT-base+Focal model exhibits poor calibration: the confidence distributions of correct and incorrect predictions are nearly identical (both centred around 0.35–0.40), and the box plots show overlapping interquartile ranges. This means the model cannot reliably distinguish when it is correct from when it is wrong — it assigns similar confidence scores to both cases. This behaviour is consistent with the train-only setting: with fewer training samples, the model is generally uncertain across all predictions. We note that this analysis applies specifically to the train-only model used for validation evaluation; the train+valid

model (Kaggle score 0.804) is expected to produce better-separated confidence distributions due to its larger training set.

8.5 Validation vs. Kaggle Gap

A consistent observation across all fine-tuned transformer models is a significant gap between validation and Kaggle scores. For DistilBERT with early stopping, the gap is 0.800 (validation) vs. 0.724 (Kaggle) — a difference of 0.076. This gap arises from **optimistic bias**: using the validation set for early stopping selection means the reported validation score reflects the best checkpoint for that specific split, not generalisation to unseen data. Training on train+valid eliminates this bias (no validation score available, but Kaggle improves to 0.761).

9 Conclusions

9.1 What Worked

- **BERT-base fine-tuning on train+valid**: Training on the full labelled data (avoiding validation leakage) improved Kaggle scores by approximately 0.03–0.05 across all models.
- **Focal Loss** ($\gamma = 2.0$, **no class weights**): The single most impactful improvement, yielding +0.043 for BERT-base.
- **Probability ensemble (soft voting)**: Ensembling heterogeneous models (BERT-base + Multi-task BERT) outperformed any individual model.
- **Multi-task learning**: Training hazard and product jointly with a shared encoder improves representation quality and provides diversity for ensemble.
- **TF-IDF + LinearSVC**: Remains a strong classical baseline (0.741) that outperforms all word-embedding methods and even some transformer approaches on this domain.

9.2 What Didn't Work

- **Word embeddings**: Mean-pooled Word2Vec/GloVe/FastText representations lose discriminative domain terminology.
- **Dimensionality reduction (LSA, BM25)**: Either removes or distorts the precise domain signal required for classification.
- **Data augmentation (EDA)**: WordNet-based augmentation corrupts domain-specific terms.
- **Label smoothing + Focal Loss**: Redundant regularisation over-penalises minority classes.
- **DeBERTa-v3**: NaN issues from tokenizer incompatibility; when resolved, poor performance due to mismatched initialisation weights.

9.3 Key Takeaways

1. **Domain-specific vocabulary is highly discriminative**. Chi-square analysis shows that classes are separated by precise technical terms (pathogen names, chemical compounds) rather than latent semantic structure. This is why TF-IDF outperforms embeddings and LSA.
2. **Focal Loss outperforms cross-entropy for imbalanced data**. But it should not be combined with class weights, as both address the same imbalance simultaneously.

3. **Ensemble diversity beats same-model averaging.** Three same-architecture models with different seeds provide minimal improvement. Heterogeneous architectures (different loss functions, different training objectives) are necessary for effective ensembling.
4. **Validation leakage inflates early-stopping scores.** The gap between validation and Kaggle scores can be as large as 0.08. Training on all available labelled data (train+valid) and using the Kaggle leaderboard as the true evaluation is more reliable.

References

- [1] Caruana, R. (1997). *Multitask learning*. Machine Learning, 28(1), 41–75.
- [2] Cortes, C., & Vapnik, V. (1995). *Support-vector networks*. Machine Learning, 20(3), 273–297.
- [3] Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of deep bidirectional transformers for language understanding*. In *Proceedings of NAACL-HLT*, 4171–4186.
- [4] He, P., Liu, X., Gao, J., & Chen, W. (2021). *DeBERTa: Decoding-enhanced BERT with disentangled attention*. In *Proceedings of ICLR*.
- [5] Hochreiter, S., & Schmidhuber, J. (1997). *Long short-term memory*. Neural Computation, 9(8), 1735–1780.
- [6] Joachims, T. (1998). *Text categorization with support vector machines: Learning with many relevant features*. In *Proceedings of ECML*, 137–142.
- [7] Kim, Y. (2014). *Convolutional neural networks for sentence classification*. In *Proceedings of EMNLP*, 1746–1751.
- [8] Lin, T.-Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). *Focal loss for dense object detection*. In *Proceedings of ICCV*, 2980–2988.
- [9] Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., ... & Stoyanov, V. (2019). *RoBERTa: A robustly optimized BERT pretraining approach*. arXiv preprint arXiv:1907.11692.
- [10] Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., ... & Liu, P. J. (2020). *Exploring the limits of transfer learning with a unified text-to-text transformer*. Journal of Machine Learning Research, 21(140), 1–67.
- [11] Rennie, J. D. M., Shih, L., Teevan, J., & Karger, D. R. (2003). *Tackling the poor assumptions of naive bayes text classifiers*. In *Proceedings of ICML*, 616–623.
- [12] Robertson, S. E., & Walker, S. (1994). *Some simple effective approximations to the 2-Poisson model for probabilistic weighted retrieval*. In *Proceedings of SIGIR*, 232–241.
- [13] Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter*. arXiv preprint arXiv:1910.01108.
- [14] Szegedy, C., Vanhoucke, V., Ioffe, S., Shlens, J., & Wojna, Z. (2016). *Rethinking the inception architecture for computer vision*. In *Proceedings of CVPR*, 2818–2826.
- [15] Vallat, M., Thordsen, E., Schindler, E., & Tran, V. (2025). *SemEval-2025 Task 9: The Food Hazard Detection Challenge*. In *Proceedings of SemEval*.
- [16] Wei, J., & Zou, K. (2019). *EDA: Easy data augmentation techniques for boosting performance on text classification tasks*. In *Proceedings of EMNLP-IJCNLP*, 6382–6388.
- [17] Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., ... & Rush, A. M. (2020). *Transformers: State-of-the-art natural language processing*. In *Proceedings of EMNLP (Systems Demonstrations)*, 38–45.
- [18] Wolpert, D. H. (1992). *Stacked generalization*. Neural Networks, 5(2), 241–259.
- [19] HuggingFace. (2024). *Transformers Documentation: Fine-tuning a pretrained model*. Retrieved from <https://huggingface.co/docs/transformers/training>

- [20] HuggingFace. (2024). *BERT base model (uncased) — Model Card*. Retrieved from <https://huggingface.co/bert-base-uncased>
- [21] Kaggle. (2025). *NLP-053-CSE-UOI-2026: Food Hazard Detection Competition*. Retrieved from <https://www.kaggle.com/competitions/nlp-053-cse-uoi-2026-food-hazard-detection>
- [22] GeeksforGeeks. (2024). *Focal Loss in Machine Learning*. Retrieved from <https://www.geeksforgeeks.org/focal-loss-in-machine-learning/>
- [23] GeeksforGeeks. (2024). *Understanding TF-IDF (Term Frequency-Inverse Document Frequency)*. Retrieved from <https://www.geeksforgeeks.org/understanding-tf-idf-term-frequency-inverse-document-frequency/>
- [24] Scikit-learn Developers. (2024). *scikit-learn: Machine Learning in Python — User Guide*. Retrieved from https://scikit-learn.org/stable/user_guide.html
- [25] PyTorch Developers. (2024). *PyTorch Documentation: torch.nn — Loss Functions*. Retrieved from <https://pytorch.org/docs/stable/nn.html>
- [26] Alammam, J. (2018). *The Illustrated BERT, ELMo, and co. (How NLP Cracked Transfer Learning)*. Retrieved from <https://jalammar.github.io/illustrated-bert/>
- [27] Sarkar, D. (2021). *Text Classification using Machine Learning* [Video]. YouTube. Retrieved from <https://www.youtube.com/watch?v=VtRLrQ3Ev-U>
- [28] Karpathy, A. (2023). *Let's build GPT: from scratch, in code, spelled out* [Video]. YouTube. Retrieved from <https://www.youtube.com/watch?v=kCc8FmEb1nY>